

TP 1 : Analyse Syntaxique Java avec Tree-sitter

Objectifs du TP

Ce TP a pour but de vous initier à l'analyse syntaxique automatique en utilisant Tree-sitter, un parseur moderne utilisé dans plusieurs éditeurs de code comme VSCode ou GitHub Copilot. Vous manipulerez un AST (arbre de syntaxe) réel produit à partir d'un code Java, et vous en extrairez des informations structurées.

Prérequis

Documentation

<https://tree-sitter.github.io/tree-sitter/>

<https://github.com/tree-sitter/tree-sitter>

<https://pypi.org/project/tree-sitter/>

Installez les dépendances suivantes dans votre environnement Python :

```
pip install tree-sitter tree-sitter-java
```

Partie 1 – Initialisation et parsing simple

1.1 Code source Java à analyser

Recopiez ou utilisez le code suivant dans une chaîne de caractères :

```
public class Test {  
    public static void main(String[] args) {  
        int x = 3 + 4 * 5;  
        System.out.println(x);  
    }  
}
```

1.2 Initialiser Tree-sitter

Utilisez le code suivant pour configurer le parseur :

```
from tree_sitter import Language, Parser  
import tree_sitter_java as tsjava  
  
JAVA_LANGUAGE = Language(tsjava.language())  
parser = Parser(JAVA_LANGUAGE)
```

1.3 Générer l'AST

Utilisez `parser.parse(...)` pour générer l'arbre syntaxique, puis affichez-le en S-expression avec `tree.root_node.sexp()`.

Partie 2 – Exploration de l'arbre de syntaxe

2.1 Parcours récursif

Créez une fonction `explore(node, code_bytes, indent=0)` qui affiche le type de chaque nœud et le code source correspondant. Utilisez :

- `node.type` : type du nœud
- `node.children` : liste des enfants
- `node.start_byte, node.end_byte` : pour extraire le texte du code

Affichez l'arbre avec indentation pour comprendre la hiérarchie des nœuds.

Partie 3 – Extraction d'opérations binaires

3.1 Objectif

Vous devez parcourir l'AST et extraire toutes les expressions binaires (ex : `3 + 4`, `4 * 5`).

3.2 Aide : fonctions utiles

- `node.type == 'binary_expression'`
- `node.child_by_field_name('left')`
- `node.child_by_field_name('right')`
- `node.children[1]` pour l'opérateur central (+, *, etc.)

3.3 Fonction à implémenter

Créez la fonction suivante :

```
def extract_binary_operations(node, code_bytes, ops=[]):  
    # votre code ici
```

Elle doit retourner une liste de dictionnaires :

```
[  
    {'op': '+', 'left': '3', 'right': {'op': '*', 'left': '4', 'right':  
    '5'}},  
]
```

Partie 4 – Questions de compréhension

1. Quelle est la différence entre l'arbre produit par Tree-sitter et un AST classique utilisé en compilation ?
2. Pourquoi Tree-sitter est-il utile dans les outils de développement moderne (IDE, analyse statique) ?
3. Quels types d'erreurs peut-on détecter en parcourant un AST ?

Bonus (facultatif)

- Ajoutez le support des parenthèses : $(2 + 3) * 4$
- Affichez l'AST au format JSON
- Marquez les expressions dont les opérandes sont des variables